Experiment #08

Parallel Implementation of Dot Product using OpenMP

Objective:

Objective of this lab is to implement dot product of vectors using parallel computation technique (OpenMP).

Software Tools:

- Ubuntu

Theory:

OpenMP, Open Multi-Processing, is a set of compiler directives and related clauses, an application programming interface (API), and environment variables that allow you to easily create applications that execute on multiple threads. The Open MP model allows you to complete the following tasks:

- Define parallel regions of code and create teams of threads that execute the parallel regions.
- Specify how to share work among the threads in the team.
- Declare data that is shared among the threads and data that is private to each thread.
- Synchronize threads, protect shared data from concurrent access, and define regions that execute by a single thread exclusively.

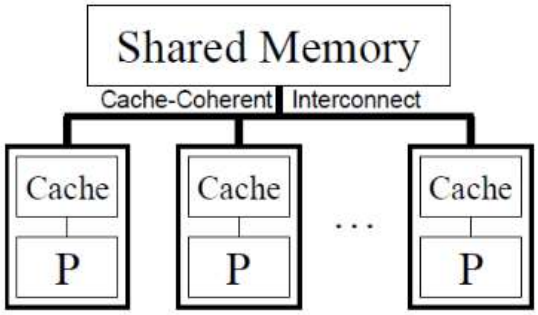
The compiler directives and associated clauses address the high level work of defining parallel regions, dividing code and loop iterations among threads, and synchronizing work among the threads of a team. The API provides a finer level of control within a parallel region; you can determine and set the number of available threads, set and initialize locks, and more. The environment variables control the execution of the parallel regions at run time.

The sections construct is a noniterative work-sharing construct that contains a set of structured blocks that are to be distributed among and executed by the threads in a team. Each structured block is executed once by one of the threads in the team in the context of its implicit task.

Parallel Memory Architectures

Before getting deep into OpenMP, let's revive the basic parallel memory architectures.
These are divided into three categories;

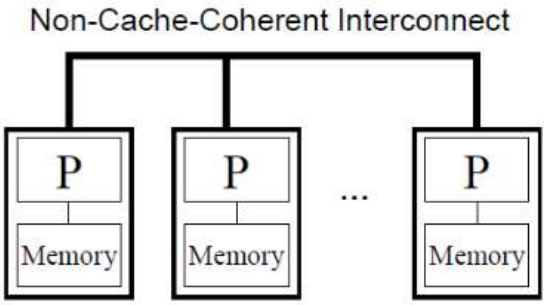
- **Shared Memory**



- Shared memory system
- For example, a single node on a cluster
- Open Multi-processing (OpenMP)

OpenMP comes under the shared memory concept. In this, different CPU's (processors) will have access to the same memory location. Since all CPU's connect to the same memory, memory access should be handled carefully.

- **Distributed Memory**



- Distributed memory system
- For example, mutli nodes on a cluster
- Message Passing Interface (MPI)

Here, each CPU (processor) will have its own memory location to access and use. In order to make them communicate, all independent systems will be connected together using a network. MPI is based on distributed architecture.

Example (Serial Execution): -

```cpp
#include <iostream>
#include <vector>
#include"Timer.h"
#include <omp.h>

using namespace std;
const int N = 1000;
int main()
{
vector<vector<int>> A(N, vector<int>(N));
vector<vector<int>> B(N, vector<int>(N));
vector<vector<int>> C(N, vector<int>(N));

Timer seqTime("Sequential Execution: ");
// Initialize random values
for (int i = 0; i < N; i++)
{
    for (int j = 0; j < N; j++)
    {
        A[i][j] = rand() % 100;
        B[i][j] = rand() % 100;
    }
}
seqTime.Start();
for (int i = 0; i < N; i++)
{
    for (int j = 0; j < N; j++)
    {
        int sum = 0;
        for (int k = 0; k < N; k++)
        {
            sum += A[i][k] * B[k][j];
        }
        C[i][j] = sum;
    }
}
seqTime.Stop();
seqTime.Print();
    return 0;
}
```

Example (Parallel Execution): -

```cpp
#include <iostream>
#include <vector>
#include"Timer.h"
#include <omp.h>

using namespace std;
const int N = 1000;
int main()
{
vector<vector<int>> A(N, vector<int>(N));
vector<vector<int>> B(N, vector<int>(N));
vector<vector<int>> C(N, vector<int>(N));

Timer parTime("Parallel Execution: ");
// Initialize random values
for (int i = 0; i < N; i++)
{
    for (int j = 0; j < N; j++)
    {
        A[i][j] = rand() % 100;
        B[i][j] = rand() % 100;
    }
}
// Perform dot product in parallel using OpenMP
parTime.Start();
   #pragma omp parallel for
   for (int i = 0; i < N; i++)
   {
        for (int j = 0; j < N; j++)
        {
            int sum = 0;
            for (int k = 0; k < N; k++)
            {
                sum += A[i][k] * B[k][j];
            }
            C[i][j] = sum;
        }
    }
parTime.Stop();
parTime.Print();
    return 0;
}
```

Output: -


fasih@fasih-Latitude-7280: ~/pdclabs/lab08
fasih@fasih-Latitude-7280:~$ cd pdclabs
fasih@fasih-Latitude-7280:~/pdclabs$ cd lab08
fasih@fasih-Latitude-7280:~/pdclabs/lab08$ subl serial.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab08$ g++ -o serial.out -fopenmp serial.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab08$ ./serial.out
Sequential Execution: : 10329.8 msec
fasih@fasih-Latitude-7280:~/pdclabs/lab08$ subl parallel.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab08$ g++ -o parallel.out -fopenmp parallel.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab08$ ./parallel.out
Parallel Execution: : 5348.75 msec
fasih@fasih-Latitude-7280:~/pdclabs/lab08$

Lab Task:

- Write a program, which compute dot product of two vectors. Calculate speedup by generating 1, 2, 4, 6 and 8 threads & draw graph between serial & parallel execution to visualize the speedup.
